VOLUME 10 OF 10

Graduate Gaming Hardware and Research Thesis

Master's-level specialization / thesis

Evaluate advanced architectures, physical design and verification, then complete and defend a research artifact.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 46 Advanced computer architecture for gaming workloads

46.1 Cache coherence, memory consistency and synchronization • 46.2 Out-of-order execution, speculation and precise state • 46.3 Architecture simulation, traces and model validation • 46.4 Memory systems, chiplets and multi-objective design

Module 47 VLSI, timing and physical implementation

47.1 CMOS logic, transistor sizing and delay models • 47.2 Static timing analysis, clocks and timing exceptions • 47.3 Synthesis, placement, clock trees, routing and physical checks • 47.4 Power, area, process corners and design-space closure

Module 48 Verification, reliability and hardware trust boundaries

48.1 Formal properties, abstraction and assume-guarantee reasoning • 48.2 Clock/reset crossings and asynchronous FIFO verification • 48.3 Fault models, ECC, testability and recovery • 48.4 Hardware security, thermal limits and assurance review

Module 49 Master's thesis: proposal, implementation and evaluation

49.1 Question selection, literature map and thesis scope • 49.2 Proposal review, baselines and risk-reduction milestones • 49.3 Implementation, ablations and controlled evaluation • 49.4 Thesis writing, reproducibility and contribution boundaries

Module 50 Thesis defense, professional practice and continued development

50.1 Oral defense and independent artifact reproduction • 50.2 Mature-codebase integration, review and maintainable contributions • 50.3 Portfolio, technical interviews and role-specific evidence • 50.4 Long-term learning, maintenance and final handoff

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 46

Advanced computer architecture for gaming workloads

Prerequisites Modules 34, 37-40 and 41-42.

Four lessons

46.1 Cache coherence, memory consistency and synchronization

Separate agreement about a location from ordering across locations.

46.2 Out-of-order execution, speculation and precise state

Increase instruction overlap without changing architectural outcomes.

46.3 Architecture simulation, traces and model validation

Use simulation as evidence with a stated fidelity boundary.

46.4 Memory systems, chiplets and multi-objective design

Choose an architecture across performance, power, cost and complexity.

Module project

Perform an architecture study using a pinned simulator or a carefully bounded analytical model. Compare at least three configurations on representative and adverse game-related workloads, and validate the model against a known reference where possible.

Core references

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R38 gem5 project — gem5 documentation

R35 RISC-V International — RISC-V ISA specifications

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 46.1 • MODULE 46

Cache coherence, memory consistency and synchronization

Learning goal Separate agreement about a location from ordering across locations.

Cache coherence governs how cached copies of a location interact, including ownership and visibility of writes. Memory consistency defines which orderings of operations across locations are observable. A coherent machine can still expose behaviors disallowed by a stronger consistency model. Programming-language memory rules add another layer between source code and hardware.

Protocols such as MESI use states and messages to preserve coherence invariants, but real implementations include transient states and races among requests. A useful invariant is that no two participants simultaneously hold conflicting write ownership under the protocol. Directory designs track sharers while snooping designs observe suitable interconnect traffic. False sharing and invalidation storms are performance effects of otherwise correct coherence. Use small litmus tests and state models to connect abstract rules to software synchronization.

WORKED EXAMPLE

A producer writes data and then publishes a flag. Correct consumer behavior depends on an appropriate release/acquire or equivalent synchronization contract; coherent caches alone do not justify ordinary unsynchronized C++ access.

PRACTICE + PASS CRITERION

Model a small coherence protocol and evaluate message-passing litmus tests under stated language and hardware models. Pass: ownership invariants hold, transient races are considered and coherence is not confused with data-race freedom.

ASSIGNED STUDY

R48: multiprocessors, coherence and consistency; R35: selected memory-ordering specification; R11: language-level concurrency.

CHECK YOUR UNDERSTANDING

1. How do coherence and consistency differ?

2. Do coherent caches make unsynchronized C++ shared access safe?

3. Why include transient states in a protocol model?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 46.2 • MODULE 46

Out-of-order execution, speculation and precise state

Learning goal Increase instruction overlap without changing architectural outcomes.

Out-of-order processors schedule ready operations while older operations may be waiting. Register renaming removes false name dependencies such as write-after-read and write-after-write, but true data dependencies remain. A reorder or retirement mechanism preserves the required architectural order and supports precise exceptions.

Branch prediction and memory speculation create work before all conditions are known. Incorrect speculation must discard inappropriate architectural effects, while microarchitectural traces may still matter to security and performance. More execution width helps only when useful parallelism, instruction supply and memory behavior support it. Analyze queue capacity, issue limits and dependency chains rather than assuming a larger core always improves a latency-sensitive game thread.

WORKED EXAMPLE

For two instructions that both write architectural register r1 but whose values are consumed separately, renaming can assign different physical destinations. A later instruction that truly consumes the first result still waits for that result.

PRACTICE + PASS CRITERION

Use a simulator or detailed scheduling model to compare in-order and out-of-order execution on independent and dependent instruction streams. Pass: architectural results agree and bottleneck explanations include true dependencies and resource limits.

ASSIGNED STUDY

R48: superscalar, out-of-order, prediction and exception handling; R38: CPU models and configuration.

CHECK YOUR UNDERSTANDING

1. Which dependencies does register renaming primarily remove?

2. Why retire architectural effects in a controlled order?

3. Why may a wider core fail to improve one game thread?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 46.3 • MODULE 46

Architecture simulation, traces and model validation

Learning goal Use simulation as evidence with a stated fidelity boundary.

A functional simulator checks what computation does; a timing model estimates when events occur under modeled resources. A cycle-oriented model is not automatically accurate for every real device. Workload inputs, cache warmup, sampled regions and operating-system effects influence results. Trace-driven studies can lose feedback between timing and program behavior.

Validate simple cases against analytical expectations and available measured hardware before trusting complex comparisons. State which pipeline, memory and interconnect mechanisms are modeled or omitted. Sensitivity analysis varies uncertain parameters to see whether conclusions survive. Preserve simulator revision, configuration and workload binaries. Relative comparisons can still be misleading when a model error favors one design disproportionately.

WORKED EXAMPLE

If a simulated memory latency is uncertain, sweep a plausible declared range. A proposed design that wins only at one finely chosen value has a weaker claim than one that remains useful across the range.

PRACTICE + PASS CRITERION

Run a cache, scheduling or accelerator study with at least one validation case and a sensitivity sweep. Pass: raw configurations reproduce results, modeled versus measured quantities are labeled and conclusions do not exceed the tested fidelity.

ASSIGNED STUDY

R38: simulation models, statistics and configuration; R48: architecture evaluation; R43: analytical bounds.

CHECK YOUR UNDERSTANDING

1. Does cycle-oriented simulation guarantee real-hardware accuracy?

2. Why perform sensitivity analysis?

3. What can a trace-driven model omit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 46.4 • MODULE 46

Memory systems, chiplets and multi-objective design

Learning goal Choose an architecture across performance, power, cost and complexity.

Memory bandwidth, latency, capacity and energy are distinct design dimensions. More channels or wider interfaces can improve bandwidth while increasing pins, routing, power and controller complexity. Cache size changes miss behavior but also area and access characteristics. Packaging and chiplet approaches introduce interconnect and partitioning tradeoffs; no packaging label guarantees a superior system.

A Pareto frontier contains designs not dominated on every chosen objective. Define constraints and compare alternatives at consistent workload quality and manufacturing assumptions. Cost and yield models require credible inputs; educational estimates should not be presented as production quotations. Include verification and software complexity in the design discussion because a theoretically efficient architecture may be impractical to support.

WORKED EXAMPLE

Design A takes 10 ms at 40 J per task; B takes 8 ms at 50 J. Neither dominates the other on latency and energy. A third design at 12 ms and 60 J is dominated by both under those two metrics.

PRACTICE + PASS CRITERION

Construct a Pareto comparison of at least three architecture configurations. Pass: objectives and assumptions are explicit, dominated designs are identified correctly and a recommendation changes appropriately when a hard constraint changes.

ASSIGNED STUDY

R48: memory and quantitative design; R49: packaging and system tradeoffs; R38: architecture alternatives.

CHECK YOUR UNDERSTANDING

1. Does A dominate B in the example?

2. Why is the 12 ms, 60 J design dominated?

3. Why not present an educational chip-cost model as a quote?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 47

VLSI, timing and physical implementation

Prerequisites Modules 31-34, 40 and 46. Use educational libraries and licensed or open tools only.

Four lessons

47.1 CMOS logic, transistor sizing and delay models

Connect gate-level choices to electrical load and timing.

47.2 Static timing analysis, clocks and timing exceptions

Check every required path under the correct timing contract.

47.3 Synthesis, placement, clock trees, routing and physical checks

Understand how a logical netlist becomes a physical layout.

47.4 Power, area, process corners and design-space closure

Report PPA with enough assumptions to make comparison meaningful.

Module project

Carry a verified small RTL block through an educational synthesis and physical-design flow. Report constraints, timing, area and power-estimation assumptions. The result is a learning artifact, not fabrication-ready commercial signoff.

Core references

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R39 OpenROAD project — OpenROAD documentation

R30 MIT OpenCourseWare — 6.002 Circuits and Electronics

R33 YosysHQ — Yosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 47.1 • MODULE 47

CMOS logic, transistor sizing and delay models

Learning goal Connect gate-level choices to electrical load and timing.

CMOS gates charge and discharge capacitances through transistor networks. Propagation delay depends on drive strength, load, input transition, topology and process/voltage/temperature conditions. Increasing device size can reduce effective resistance while increasing input capacitance and area, moving cost to the previous stage.

Logical-effort-style models separate gate topology, electrical effort and parasitic delay to reason about sizing and staging. These are approximations, not substitutes for characterized library timing. Series transistor stacks and fanout affect delay differently. Dynamic switching, leakage and short-circuit effects contribute to power. Study an inverter chain and a small logic path before optimizing a large synthesized block.

WORKED EXAMPLE

In the simplified normalized model d=g*h+p, a stage with g=1, h=4 and p=1 has delay 5 units. Those normalized units require a technology-dependent scale before becoming nanoseconds.

PRACTICE + PASS CRITERION

Analyze and simulate an inverter chain or compare library-cell drive strengths under several loads. Pass: assumptions are stated, input loading is included and model predictions are compared with available characterized or simulated results.

ASSIGNED STUDY

R49: CMOS devices, combinational circuits and delay; R30: MOS circuit foundations.

CHECK YOUR UNDERSTANDING

1. What is d for g=1, h=4, p=1?

2. Why can enlarging one gate slow an earlier stage?

3. Can normalized logical-effort delay be reported directly as nanoseconds?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 47.2 • MODULE 47

Static timing analysis, clocks and timing exceptions

Learning goal Check every required path under the correct timing contract.

Static timing analysis evaluates path delays without enumerating every input waveform. Setup checks constrain maximum data arrival before capture; hold checks constrain minimum delay around the capture edge. Clock-to-output, combinational delay, setup/hold requirements, skew and uncertainty all participate. Different corners can dominate different checks.

Clocks, generated clocks, input/output delays and exceptions define what the tool analyzes. An unconstrained path is not a fast path; it may be an unexamined path. False-path and multicycle exceptions must follow real functional guarantees, not serve as a way to hide violations. Review exception reachability and clock-domain boundaries. Functional correctness and timing correctness require separate evidence.

WORKED EXAMPLE

With zero skew and uncertainty, tCQ_max=0.10 ns, logic_max=0.50 ns and setup=0.05 ns require T>=0.65 ns. Real signoff adds the relevant clock, interconnect and corner details.

PRACTICE + PASS CRITERION

Constrain a small sequential block and inspect worst setup and hold paths. Pass: no unexplained unconstrained endpoints, every exception is justified and a deliberately tightened clock produces an understood violation.

ASSIGNED STUDY

R39: timing analysis and flow constraints; R49: sequential timing and clocking.

CHECK YOUR UNDERSTANDING

1. What minimum simplified period follows from the example?

2. Does marking a path false make a real timing problem disappear physically?

3. Why examine both maximum and minimum delays?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 47.3 • MODULE 47

Synthesis, placement, clock trees, routing and physical checks

Learning goal Understand how a logical netlist becomes a physical layout.

The implementation flow maps logic, places cells, constructs clock distribution and routes interconnect under design constraints. Placement changes wire length and congestion; routing adds parasitic resistance and capacitance that affect timing and power. Clock-tree design must distribute a usable clock while controlling skew and other requirements.

Physical checks include geometry rules and connectivity consistency between layout and the intended circuit. Passing one tool stage does not establish all manufacturing, electrical or reliability requirements. Educational process kits and libraries have specific scope and licensing. Preserve exact versions and distinguish pre-route estimates from extracted post-route results. A failed route or timing closure attempt is useful evidence about congestion or unrealistic constraints.

WORKED EXAMPLE

A netlist can meet an idealized pre-route timing estimate and fail after long interconnect is included. The logical cell count alone does not reveal that physical bottleneck.

PRACTICE + PASS CRITERION

Run a small block through an educational OpenROAD-compatible flow or inspect a provided open example. Pass: capture stage reports, identify a routing/timing change and state which physical checks were and were not performed.

ASSIGNED STUDY

R39: flow stages, placement, clock tree and routing; R49: physical-design methodology and EDA.

CHECK YOUR UNDERSTANDING

1. Why can timing worsen after routing?

2. Does synthesis success prove a manufacturable layout exists?

3. Why preserve process-kit and library versions?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 47.4 • MODULE 47

Power, area, process corners and design-space closure

Learning goal Report PPA with enough assumptions to make comparison meaningful.

Power-performance-area evaluation depends on workload activity, clocking, voltage, cell library and physical implementation. A common simplified dynamic-power model is P=alpha*C*V^2*f, where alpha represents switching activity under a stated convention. Leakage and other contributions are additional. A power estimate without activity assumptions can be badly misleading.

Process, voltage and temperature corners model variation but do not cover every reliability or manufacturing issue. Compare designs under consistent constraints and report whether timing actually closes. Clock gating and operand isolation can reduce unnecessary switching but require functional and timing verification. An educational area reduction is meaningful only alongside performance, power and correctness effects. Keep fabrication and production signoff outside the claim.

WORKED EXAMPLE

Under the simplified dynamic model with unchanged alpha, C and f, reducing voltage to 0.9 of its original value scales dynamic power to 0.9^2=0.81. In practice the achievable frequency and margins may also change.

PRACTICE + PASS CRITERION

Compare two verified RTL implementations using identical flow assumptions. Pass: report area, worst timing and activity-based power estimates where supported, with uncertainty and no unsupported commercial-silicon claim.

ASSIGNED STUDY

R49: power, timing and implementation tradeoffs; R39: reports and analysis; R30: energy and electrical models.

CHECK YOUR UNDERSTANDING

1. What dynamic-power factor follows from 0.9x voltage under fixed other factors?

2. Why is an activity assumption necessary for power estimation?

3. Why can lower voltage invalidate a timing target?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 48

Verification, reliability and hardware trust boundaries

Prerequisites Modules 33-35, 46-47.

Four lessons

48.1 Formal properties, abstraction and assume-guarantee reasoning

Prove useful properties without hiding failures in assumptions.

48.2 Clock/reset crossings and asynchronous FIFO verification

Check digital protocol correctness while acknowledging analog risk.

48.3 Fault models, ECC, testability and recovery

Design for detectable failures rather than assuming perfect hardware.

48.4 Hardware security, thermal limits and assurance review

Include isolation and failure containment in the design contract.

Module project

Create a verification dossier for the hardware capstone: requirements-to-tests traceability, assertions, coverage, clock/reset review, fault injection and a clearly bounded assurance claim.

Core references

R50 YosysHQ — SymbiYosys documentation

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R39 OpenROAD project — OpenROAD documentation

R51 Linux kernel contributors — Driver implementer API guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 48.1 • MODULE 48

Formal properties, abstraction and assume-guarantee reasoning

Learning goal Prove useful properties without hiding failures in assumptions.

Formal verification reasons about a mathematical model and properties under assumptions. Decompose a large system into contracts: a block assumes valid inputs and guarantees specified outputs or progress. The system review must show that connected blocks actually satisfy each other's assumptions. Circular or inconsistent contracts can make local proofs meaningless.

Bounded model checking, induction and equivalence checking answer different questions. A timeout is not a proof or a counterexample. Vacuity checks and cover properties help reveal environments that never exercise the intended behavior. Abstraction can make proof tractable but must conservatively preserve the property of interest. Track proof scripts, tool versions and unresolved properties as part of the artifact.

WORKED EXAMPLE

An arbiter property stating that every request is eventually granted becomes vacuous if the environment assumes requests are always zero. A cover goal requiring contention exposes that missing behavior.

PRACTICE + PASS CRITERION

Prove a queue, arbiter or transaction property and inject a violation. Pass: assumptions are reviewed independently, contention is covered and bounded, proven, failed and unknown results are reported separately.

ASSIGNED STUDY

R50: safety, liveness, induction, cover and assumptions; R49: verification methodology.

CHECK YOUR UNDERSTANDING

1. Is a solver timeout evidence that the property is true?

2. Why can the no-request assumption make an arbiter proof vacuous?

3. What must assume-guarantee integration establish?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 48.2 • MODULE 48

Clock/reset crossings and asynchronous FIFO verification

Learning goal Check digital protocol correctness while acknowledging analog risk.

Clock-domain crossings require both a sound transfer protocol and an appropriate physical synchronizer implementation. Asynchronous FIFOs often use local binary pointers with Gray-coded values transferred between domains, because adjacent Gray values change one bit. Correct full/empty detection, pointer widths, reset behavior and synchronization delay still need verification.

Gray coding is not a substitute for routing/skew constraints or correct sampling assumptions. Multi-bit coherence and pulse capture need distinct reasoning. Reset deassertion must be coordinated with each domain's requirements, and resetting one side of a live link needs a recovery policy. Digital formal analysis proves the modeled protocol; analog metastability probability needs device and timing information beyond that model.

WORKED EXAMPLE

A one-bit source pulse can occur entirely between two destination clock edges and be missed. A level handshake or suitable event-transfer scheme is needed when every event must be observed.

PRACTICE + PASS CRITERION

Model an asynchronous FIFO or event handshake with independent clocks and reset sequences. Pass: no loss/duplication under the stated protocol, full/empty behavior is checked and physical synchronizer assumptions are listed separately.

ASSIGNED STUDY

R50: multiclock modeling and verification; R49: sequential circuits and synchronization; revisit Module 32 timing boundaries.

CHECK YOUR UNDERSTANDING

1. Why is Gray code useful for pointer transfer?

2. Can an unsynchronized one-cycle pulse be missed by another domain?

3. Does a digital FIFO proof quantify real metastability failure rate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 48.3 • MODULE 48

Fault models, ECC, testability and recovery

Learning goal Design for detectable failures rather than assuming perfect hardware.

A fault model specifies the failures considered, such as stuck-at logic, transient bit flips or communication corruption. Detection and correction capability depend on the code or mechanism and the fault pattern. Parity detects an odd number of flipped bits but does not locate and correct an arbitrary single error by itself. Error-correcting codes add structured redundancy with explicit guarantees.

Design-for-test techniques improve controllability and observability of internal state, with area, timing and security implications. Built-in self-test and scan are not interchangeable with complete functional verification. A detected error needs policy: retry, isolate, reset, degrade or fail safely. Fault injection tests the policy as well as detection. Report unmodeled common-mode or multi-bit failures instead of assuming independence without evidence.

WORKED EXAMPLE

With one parity bit over a word, a single flipped data bit changes parity and is detectable. Two flipped bits can leave parity unchanged, so the same scheme cannot guarantee detection of every double-bit error.

PRACTICE + PASS CRITERION

Implement a parity and a small single-error-correcting code model, then inject faults. Pass: enumerate the supported correction/detection claims, test multi-bit counterexamples and define a recovery response.

ASSIGNED STUDY

R49: testing, DFT and reliability context; R51: error detection/correction reporting concepts; R06: coding and combinatorial reasoning extension.

CHECK YOUR UNDERSTANDING

1. Can simple parity detect every two-bit error?

2. Why specify a fault model before claiming reliability?

3. What is missing when a system detects an error but has no recovery policy?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 48.4 • MODULE 48

Hardware security, thermal limits and assurance review

Learning goal Include isolation and failure containment in the design contract.

Hardware interfaces control powerful resources: memory access, privilege, interrupts and firmware execution. Validate descriptors, restrict DMA and separate trusted control from untrusted data. Boot authenticity, update integrity and rollback policy require established cryptographic mechanisms and careful key handling, not a homemade cipher. Debug interfaces need a lifecycle access policy.

Shared caches, timing and power behavior can expose information even when architectural results are isolated. Study threat models and defensive mitigations without attacking systems outside an authorized lab. Thermal and electrical protections remain active during experiments. An assurance review connects threats and hazards to requirements, mechanisms, tests and residual risks. Passing a educational checklist is not security certification.

WORKED EXAMPLE

A device accepting an unchecked memory address in a work descriptor can cross the intended buffer boundary. A protected mapping and validated length constrain that authority even when the computation itself is correct.

PRACTICE + PASS CRITERION

Write a threat-and-failure review for the accelerator and test malformed descriptors, denied access and reset under load in simulation. Pass: isolation limits, protection assumptions and unresolved risks are explicit; no unauthorized target testing occurs.

ASSIGNED STUDY

R51: device access, DMA and isolation; R49: implementation and verification; R50: enforceable protocol properties.

CHECK YOUR UNDERSTANDING

1. Why validate both address and length in a work descriptor?

2. Can correct architectural output coexist with a side-channel risk?

3. Does this review constitute a security certification?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 49

Master's thesis: proposal, implementation and evaluation

Prerequisites Gate I and relevant advanced-track modules; independent technical mentorship strongly recommended.

Four lessons

49.1 Question selection, literature map and thesis scope

Turn a broad interest into a falsifiable, feasible research question.

49.2 Proposal review, baselines and risk-reduction milestones

Make failure cheap before the expensive implementation phase.

49.3 Implementation, ablations and controlled evaluation

Execute the study without changing the question to fit a favorable result.

49.4 Thesis writing, reproducibility and contribution boundaries

Write a technical argument another engineer can audit.

Module project

Prepare and execute a thesis-scale project in one primary specialization. Budget 400-800 focused hours as a planning range, with scope adjusted through review. The deliverable is a defensible contribution and reproducible artifact, not a promise of publication.

Core references

R42 Stephen Boyd and Lieven Vandenberghe — Convex Optimization

R44 Benedikt Bitterli and coauthors — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R55 Matthias Mueller and collaborators — Author publication archive: position-based simulation and XPBD

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 49.1 • MODULE 49

Question selection, literature map and thesis scope

Learning goal Turn a broad interest into a falsifiable, feasible research question.

A thesis question identifies a problem, intervention or method, evaluation setting and meaningful outcome. Narrowness improves rigor: one well-tested engine subsystem or hardware tradeoff is preferable to a promise to build an entire commercial console. Distinguish an engineering integration contribution, a new method, a replication study and a negative result.

Build a literature map with seminal work, strong baselines and relevant newer work reviewed when the thesis begins. The dated anchor papers in this curriculum are not a complete current literature review. Identify the gap and explain why it matters. Check data, hardware, tool access, licenses and mentorship before committing. Write explicit exclusions so the project remains feasible.

WORKED EXAMPLE

Instead of "make a faster engine," ask whether a bounded task-scheduling policy reduces p99 frame time under specified streaming bursts without exceeding a declared memory budget, compared with two competent baselines.

PRACTICE + PASS CRITERION

Write a 3-5 page proposal with a research question, 8-15 relevant sources, baseline plan, constraints and exclusions. Pass: a reviewer can explain what observation would weaken or refute the main hypothesis.

ASSIGNED STUDY

Use Module 41's paper method and the chosen track's original papers; R44, R48, R49 or R55 are starting anchors, not a substitute for the literature search.

CHECK YOUR UNDERSTANDING

1. What makes a research question falsifiable?

2. Why state explicit exclusions?

3. Are the curriculum's historical anchor papers a complete current literature review?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 49.2 • MODULE 49

Proposal review, baselines and risk-reduction milestones

Learning goal Make failure cheap before the expensive implementation phase.

A credible proposal includes a baseline that already runs, an evaluation protocol and a risk register. Technical risks include insufficient performance, unavailable data, unsupported tool features and an inability to measure the claimed effect. Resolve the highest-impact uncertainties with small prototypes before building a large framework.

Milestones should produce evidence: a reference result, a validated instrument, a working minimal method, an ablation and a reproducible final comparison. A calendar date alone is not a technical milestone. Define stop or pivot criteria in advance. Independent review can reveal an unfair baseline or an untestable objective. Keep project decisions in a versioned log so later changes are transparent.

WORKED EXAMPLE

Before implementing a new GPU scheduler, confirm that the target platform exposes enough timing information to distinguish scheduling delay from execution time. Otherwise the proposed mechanism may be impossible to evaluate directly.

PRACTICE + PASS CRITERION

Deliver a running baseline, risk register and milestone plan. Pass: the primary metric can be measured, a small feasibility experiment exists and each major risk has a mitigation or explicit acceptance decision.

ASSIGNED STUDY

R49: project milestone and reporting structure; Module 41: experimental design; chosen method's primary documentation.

CHECK YOUR UNDERSTANDING

1. Why require a running baseline early?

2. What makes a milestone evidence-based?

3. Why define pivot criteria before final results?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 49.3 • MODULE 49

Implementation, ablations and controlled evaluation

Learning goal Execute the study without changing the question to fit a favorable result.

Implement the smallest method that tests the research question. Keep a correct reference path and add tests before optimizing. Separate development experiments from final evaluation, and record changes to the protocol with reasons. A method that fails on some supported inputs must report those failures rather than silently removing the cases.

Ablations test the contribution of individual components. Sensitivity sweeps examine parameter dependence; held-out workloads examine generalization. Statistical analysis should match the trial structure and metric. Preserve raw data, seeds, binaries or build manifests and analysis scripts. When results are negative, investigate correctness, measurement and assumptions, then report the supported conclusion. Honest negative evidence can be a substantial contribution.

WORKED EXAMPLE

If a new allocator reduces average time but worsens p99 frame time beyond the stated requirement, the original latency hypothesis is not supported even though one secondary number improved.

PRACTICE + PASS CRITERION

Run the approved evaluation with baselines, at least one ablation and one sensitivity study. Pass: scripts regenerate results, failed cases remain visible and claims use the preregistered primary metric or clearly disclose deviations.

ASSIGNED STUDY

R09 and Module 41: uncertainty; chosen track's original methods and implementation references; handbook experiment manifest.

CHECK YOUR UNDERSTANDING

1. Does improving a secondary metric prove the primary hypothesis?

2. Why retain failed supported cases?

3. What should happen after a negative result?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 49.4 • MODULE 49

Thesis writing, reproducibility and contribution boundaries

Learning goal Write a technical argument another engineer can audit.

A thesis report connects motivation, prior work, method, implementation, experiments, results and limitations. Each major claim should point to a derivation, test, table or figure. Separate observations from interpretations and specify uncertainty. Explain enough detail for a competent reader to reconstruct the central method without depending on undocumented conversations.

Include an artifact guide with environment, commands, expected outputs and resource needs. Record third-party code, assets, datasets and AI assistance with their relevant licenses and provenance. Do not include confidential material without authorization. The contribution statement should identify what is new, what is reproduced and what remains unverified. A polished document cannot compensate for missing experiments or an irreproducible implementation.

WORKED EXAMPLE

A report saying "the method is 30% faster" should specify the baseline, workload, metric, aggregation, hardware, quality constraint and uncertainty. Otherwise the number lacks an interpretable claim boundary.

PRACTICE + PASS CRITERION

Produce a thesis-style report, typically 30-60 pages plus appendices as a planning guide, and a complete artifact package. Pass: a reviewer traces five major claims to evidence and reproduces the central table or experiment.

ASSIGNED STUDY

R49: final project reports; chosen original papers for reporting comparisons; handbook thesis outline and evidence checklist.

CHECK YOUR UNDERSTANDING

1. What must accompany a speedup claim?

2. Why distinguish observation from interpretation?

3. What belongs in the contribution statement?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 50

Thesis defense, professional practice and continued development

Prerequisites Module 49 and all prerequisites for the selected specialization.

Four lessons

50.1 Oral defense and independent artifact reproduction

Show that the learner owns the reasoning and implementation.

50.2 Mature-codebase integration, review and maintainable contributions

Transfer a prototype into an engineering environment with other owners.

50.3 Portfolio, technical interviews and role-specific evidence

Present concrete engineering competence without inflated claims.

50.4 Long-term learning, maintenance and final handoff

Keep the knowledge and artifacts useful after the course ends.

Module project

Gate J: defend the thesis, pass an independent artifact reproduction, complete a maintainable handoff and publish an honest portfolio where rights permit. This completes the self-study program, not an accredited degree.

Core references

R01 ACM / IEEE-CS / AAAI — CS2023 knowledge areas

R02 ACM / IEEE Computer Society — Computer Engineering Curricula 2016

R05 Scott Chacon and Ben Straub — Pro Git

R11 C++ Core Guidelines editors — C++ Core Guidelines

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 50.1 • MODULE 50

Oral defense and independent artifact reproduction

Learning goal Show that the learner owns the reasoning and implementation.

An oral defense tests the relationship between the research question, assumptions, implementation and conclusions. Begin with the central claim and evidence, then invite scrutiny of the weakest assumptions. Be prepared to derive a key equation, explain a failed experiment and modify a small part of the implementation live. Saying that a tool generated the code is not an explanation of its correctness.

An independent reproduction uses the supplied artifact instructions without hidden files or undocumented manual repairs. Record what the reviewer could reproduce and what failed. A defense can pass with limitations when the claims are narrowed honestly; it should not pass on presentation alone while the central evidence remains unavailable. Revisions should respond directly to review findings.

WORKED EXAMPLE

A reviewer changes a supported workload parameter. The learner should predict which resource or error term may change, run the test and explain agreement or disagreement with that prediction.

PRACTICE + PASS CRITERION

Hold a 30-45 minute technical defense with a qualified reviewer or peer panel and a separate artifact run. Pass: the central result is reproduced, major questions receive evidence-based answers and required revisions are completed.

ASSIGNED STUDY

R49: design-project evaluation as a reference; handbook defense questions and rubric.

CHECK YOUR UNDERSTANDING

1. What does a live modification test beyond a prepared presentation?

2. Must every thesis result be positive to pass?

3. Why separate independent reproduction from the author's demo?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 50.2 • MODULE 50

Mature-codebase integration, review and maintainable contributions

Learning goal Transfer a prototype into an engineering environment with other owners.

A research prototype and a maintained engine component face different requirements. Integration needs style consistency, tests, platform support, migration behavior, documentation and ownership. Study an existing codebase through a narrow feature path: entry point, data flow, invariants, tests and build configuration. Avoid rewriting a subsystem before understanding why it exists.

A good contribution starts with a focused issue and a reproducible case. Discuss design where project norms require it, make a small patch and respond constructively to review. Open-source maintainers are not obligated to accept educational work. A local integration exercise can satisfy this lesson when no appropriate public contribution is available. Respect licenses and never upload employer or proprietary SDK code without authorization.

WORKED EXAMPLE

A faster container may break serialization ordering or iterator expectations elsewhere in the engine. Integration review must examine those contracts before accepting the local benchmark improvement.

PRACTICE + PASS CRITERION

Integrate one tested improvement into a mature open codebase or a maintained local fork. Pass: a focused patch, regression test, design rationale and reviewer response are included, whether or not an upstream project accepts it.

ASSIGNED STUDY

R05: distributed workflows and contributing; R11: interfaces and maintenance; R26: mature-engine comparison where authorized.

CHECK YOUR UNDERSTANDING

1. Why can a locally faster component be an integration regression?

2. Is upstream acceptance required to prove learning?

3. Why avoid a large rewrite as the first contribution?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 50.3 • MODULE 50

Portfolio, technical interviews and role-specific evidence

Learning goal Present concrete engineering competence without inflated claims.

A portfolio should show a small number of complete artifacts with clear ownership, build instructions, tests, measured results and limitations. Tailor emphasis: engine roles need runtime architecture and debugging; rendering roles need graphics mathematics and GPU evidence; platform roles need systems and latency; hardware roles need RTL, verification and timing or board evidence.

Technical interviews often probe fundamentals through unfamiliar problems, so practice explanation, assumptions, edge cases and tests rather than memorizing only known answers. Revisit algorithms, memory, concurrency and mathematics using transfer tasks. Do not label this self-study program an accredited degree or imply professional chip signoff from educational tools. Honest scope and a reproducible artifact are stronger than a list of fashionable technologies.

WORKED EXAMPLE

A portfolio entry can state: "Implemented a verified FIFO and compared two storage mappings under one pinned synthesis flow; physical silicon and production signoff were not performed." That is precise and defensible.

PRACTICE + PASS CRITERION

Prepare three portfolio case studies and complete one systems, one algorithm and one specialization mock interview. Pass: every claimed result is supported by an artifact and assistance or third-party contributions are clearly attributed.

ASSIGNED STUDY

R01-R02: breadth review; R11: engineering explanation and contracts; handbook role map and portfolio template.

CHECK YOUR UNDERSTANDING

1. What makes a portfolio claim defensible?

2. Should completing this program be described as earning an accredited master's degree?

3. Why practice unfamiliar transfer tasks?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 50.4 • MODULE 50

Long-term learning, maintenance and final handoff

Learning goal Keep the knowledge and artifacts useful after the course ends.

Engineering knowledge includes durable concepts and changing interfaces. Preserve principles such as invariants, measurement and ownership while periodically checking toolchains, specifications and platform documentation. A dependency update should run tests and benchmarks rather than assume compatibility from a version number. Maintain an evidence log of what was actually revalidated.

Create a handoff document describing architecture, build, tests, known risks and next decisions. Archive raw research data and source in accessible formats with appropriate backups. Choose a next specialization from demonstrated interest and gaps, not pressure to master every field simultaneously. The final review should identify strengths and unresolved areas honestly. Continuing with mentorship, collaboration and real projects is part of professional development, not a failure of the curriculum.

WORKED EXAMPLE

Updating a graphics SDK can change tutorial requirements or validation behavior. A pinned working environment preserves reproducibility while a separate upgrade branch tests the new toolchain.

PRACTICE + PASS CRITERION

Complete Gate J: archive the thesis artifact, produce a maintainer handoff and write a 90-day learning plan with one primary gap and one real contribution goal. Pass: another person can locate, build and understand the project without the author's machine.

ASSIGNED STUDY

R05: maintenance workflows; R01-R02: periodic breadth review; current official documentation for the selected tools when an update is undertaken.

CHECK YOUR UNDERSTANDING

1. Why keep a pinned environment while testing an upgrade separately?

2. What belongs in a maintainer handoff?

3. Why choose one primary next specialization?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.